

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA51 **COURSE NAME:** Cryptography and Network Security

COURSE OUTCOME COVERED

CO4: *Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security.CO (BL3, BL6)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A ***Coding Challenge***

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A
Coding Challenge

1. Write a program to simulate the IPSec Authentication Header (AH) process that generates and verifies a message authentication code (MAC) for IP packet integrity.
2. Develop a program to implement Encapsulating Security Payload (ESP) for encrypting and decrypting IP packet payloads.
3. Design a program to manage Security Associations (SA) in IPSec, including creation, lookup, and deletion operations.
4. Implement a simplified Pretty Good Privacy (PGP) system for secure email communication using public key encryption and digital signatures.
5. Create a program to classify emails as spam or legitimate using email header analysis and content-based filtering techniques.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints

Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases
--------------------	-----	--	-------------------------	-----------------------------	------------------------------

QUESTION – 1

Title

Simulation of IPSec Authentication Header (AH) Process Using Python

Aim

To develop a Python program that simulates the **IPSec Authentication Header (AH)** process by generating and verifying a Message Authentication Code (MAC) for an IP packet to ensure message integrity and authentication.

Objectives

1. Understand the purpose of IPSec Authentication Header (AH).
2. Generate a MAC for an IP packet.
3. Verify the integrity of the received packet.
4. Detect unauthorized modification of packet contents.
5. Demonstrate the working principle of HMAC in IPSec AH.

Theory

IPSec is a network-layer security protocol suite used to protect IP communications.

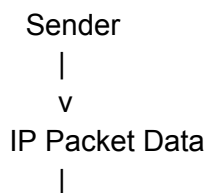
The **Authentication Header (AH)** protocol provides:

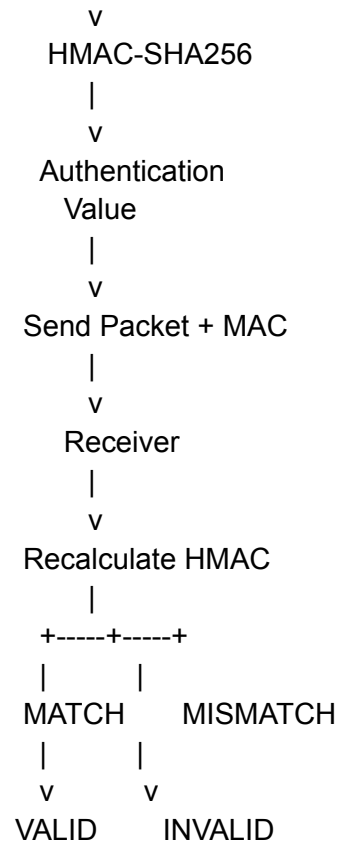
- Data integrity
- Data-origin authentication
- Protection against replay attacks when sequence numbers are used

AH does **not provide encryption**. The original IP payload remains readable.

For this simplified simulation, **HMAC-SHA-256** is used to generate the authentication value.

The process is:





Algorithm

1. Start the program.
2. Enter the IP packet data.
3. Enter or generate a secret authentication key.
4. Calculate HMAC-SHA-256 using the packet data and secret key.
5. Display the generated authentication value.
6. Simulate packet reception.
7. Recalculate the HMAC using the received packet.
8. Compare the received and calculated MAC values.
9. If they match, accept the packet.
10. If they differ, reject the packet.
11. Modify the packet to demonstrate tampering.
12. Perform verification again.
13. Stop.

Python Program

```
import hashlib  
import hmac
```

```

# -----
# Generate HMAC for IP packet
# -----
def generate_mac(packet, key):
    return hmac.new(
        key,
        packet,
        hashlib.sha256
    ).hexdigest()

# -----
# Verify IP packet
# -----
def verify_packet(packet, key, received_mac):

    calculated_mac = generate_mac(packet, key)

    return hmac.compare_digest(
        calculated_mac,
        received_mac
    )

# -----
# Main Program
# -----

print("=====")
print("    IPSec AH SIMULATION")
print("=====")

# Secret key
key = b"IPSec-Secret-Key"

# Original IP packet
packet = (
    b"SRC=192.168.1.10;"
    b"DST=192.168.1.20;"
    b"DATA=Hello Secure Network"
)

```

```

# Generate MAC
mac_value = generate_mac(packet, key)

print("\nOriginal IP Packet:")
print(packet.decode())

print("\nGenerated HMAC-SHA-256:")
print(mac_value)

# -----
# Verify original packet
# -----

print("\n--- Original Packet Verification ---")

if verify_packet(packet, key, mac_value):
    print("Authentication: SUCCESS")
    print("Integrity: VALID")
    print("Packet accepted.")
else:
    print("Authentication: FAILED")
    print("Packet rejected.")

# -----
# Simulate packet modification
# -----

modified_packet = (
    b"SRC=192.168.1.10;"
    b"DST=192.168.1.20;"
    b"DATA=Hacked Network"
)

print("\n--- Modified Packet Verification ---")

print("\nModified IP Packet:")
print(modified_packet.decode())

if verify_packet(modified_packet, key, mac_value):
    print("Authentication: SUCCESS")
    print("Integrity: VALID")
    print("Packet accepted.")
else:

```



```
print("Authentication: FAILED")
print("Integrity: INVALID")
print("Packet rejected.")
```

Sample Output

```
=====
IPSec AH SIMULATION
=====

Original IP Packet:
SRC=192.168.1.10;DST=192.168.1.20;DATA=Hello Secure Network

Generated HMAC-SHA-256:
[64-character hexadecimal authentication value]

--- Original Packet Verification ---
Authentication: SUCCESS
Integrity: VALID
Packet accepted.

--- Modified Packet Verification ---

Modified IP Packet:
SRC=192.168.1.10;DST=192.168.1.20;DATA=Hacked Network

Authentication: FAILED
Integrity: INVALID
Packet rejected.
```

The HMAC value will be different if the key or packet contents are changed.

Explanation of Code

hmac.new()

```
hmac.new(key, packet, hashlib.sha256)
```

This generates an HMAC using:

- Secret key

- Packet data
- SHA-256 hash function

hexdigest()

.hexdigest()

Converts the authentication value into a hexadecimal string.

hmac.compare_digest()

hmac.compare_digest(calculated_mac, received_mac)

Safely compares the two MAC values.

If they are equal, the packet has passed integrity verification.

Test Cases

Test Case	Packet Status	Expected Result
1	Original packet	Accepted
2	Modified payload	Rejected
3	Incorrect key	Rejected
4	Modified source address	Rejected
5	Modified destination address	Rejected

Result

The IPSec AH process was successfully simulated using Python and HMAC-SHA-256. The original packet generated a valid authentication value and was accepted. When the packet data was modified, the calculated HMAC changed and the packet was rejected.

Conclusion

The experiment demonstrates how IPSec AH can provide **data integrity and authentication**. Any modification to the authenticated packet causes the authentication value to change, allowing the receiver to detect tampering.

AH provides authentication and integrity but **does not encrypt the IP payload**. In real IPSec implementations, AH also involves additional IP-header processing and sequence-number-based anti-replay protection; this program is a simplified educational simulation.

QUESTION – 4

Title

Implementation of a Simplified PGP System for Secure Email Communication Using Python

Aim

To implement a simplified **Pretty Good Privacy (PGP)** system using Python that provides secure email communication through public-key encryption and digital signatures.

Objectives

1. Understand the basic working principle of PGP.
2. Generate public and private keys.
3. Encrypt an email message.
4. Digitally sign the message.
5. Verify the digital signature.
6. Decrypt the encrypted message.
7. Demonstrate confidentiality, integrity, and authentication.

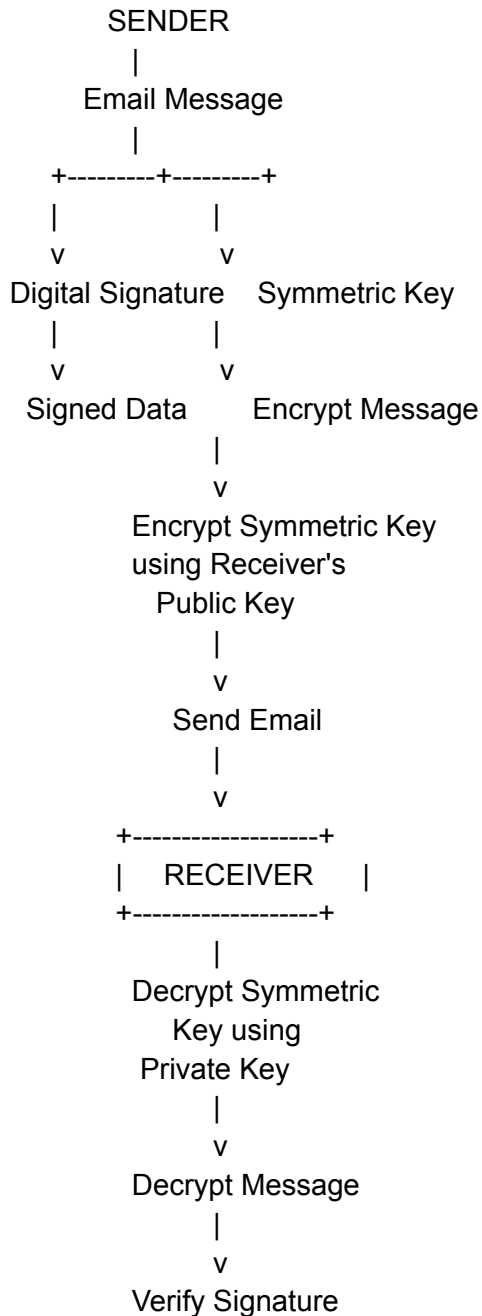
Theory

Pretty Good Privacy (PGP) is a security system used for protecting digital communications, particularly email.

PGP combines:

- Symmetric encryption
- Public-key encryption
- Cryptographic hashing
- Digital signatures

A simplified PGP process can be represented as:



PGP commonly uses a **hybrid cryptographic approach** because symmetric encryption is efficient for large messages while public-key cryptography is suitable for protecting the session key.

Cryptographic Components

Component	Purpose
RSA	Public-key encryption/signature
AES	Efficient symmetric encryption
SHA-256	Message hashing
Digital Signature	Authentication and integrity
Public Key	Encryption/verification
Private Key	Decryption/signing

Algorithm

Sender

1. Generate sender's RSA private and public keys.
2. Generate receiver's RSA private and public keys.
3. Create the email message.
4. Generate a digital signature using the sender's private key.
5. Generate a random AES session key.
6. Encrypt the message using AES.
7. Encrypt the AES key using the receiver's RSA public key.
8. Send the encrypted message, encrypted AES key, and signature.

Receiver

1. Use the receiver's RSA private key to decrypt the AES session key.
2. Use the AES session key to decrypt the email.
3. Use the sender's RSA public key to verify the digital signature.
4. Accept the message if signature verification succeeds.

Python Program

```
import os
```

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
```

```
# -----
```

```
# RSA KEY GENERATION
```

```
# -----
```

```
def generate_rsa_key_pair():
```

```
    private_key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=2048
    )
```

```
    public_key = private_key.public_key()
```

```
    return private_key, public_key
```

```
# -----
```

```
# DIGITAL SIGNATURE
```

```
# -----
```

```
def sign_message(private_key, message):
```

```
    signature = private_key.sign(
        message,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )
```

```
    return signature
```

```
# -----
```

```
# SIGNATURE VERIFICATION
```

```
# -----
```

```
def verify_signature(public_key, message, signature):
```

```
    try:
```

```
        public_key.verify(
            signature,
            message,
```

```
padding.PSS(
    mgf=padding.MGF1(hashes.SHA256()),
    salt_length=padding.PSS.MAX_LENGTH
),
hashes.SHA256()
)
```

```
return True
```

```
except Exception:
```

```
return False
```

```
# -----
# AES ENCRYPTION
# -----
```

```
def encrypt_message(message, aes_key):
```

```
    nonce = os.urandom(12)
```

```
    aes = AESGCM(aes_key)
```

```
    ciphertext = aes.encrypt(
        nonce,
        message,
        None
    )
```

```
    return nonce, ciphertext
```

```
# -----
# AES DECRYPTION
# -----
```

```
def decrypt_message(nonce, ciphertext, aes_key):
```

```
    aes = AESGCM(aes_key)
```

```
    plaintext = aes.decrypt(
        nonce,
        ciphertext,
```

```

        None
    )

    return plaintext

# -----
# MAIN PGP SIMULATION
# -----

print("=====")
print("    SIMPLIFIED PGP EMAIL SYSTEM")
print("=====")

# Generate sender keys
sender_private, sender_public = generate_rsa_key_pair()

# Generate receiver keys
receiver_private, receiver_public = generate_rsa_key_pair()

# -----
# Email Message
# -----

message = b"""
From: alice@example.com
To: bob@example.com
Subject: Secure Communication

Hello Bob,
This is a confidential PGP protected email.
"""

print("\nOriginal Email:")
print(message.decode())

# -----
# Step 1: Sign the Message
# -----

```



```

signature = sign_message(
    sender_private,
    message
)

print("\nDigital Signature Generated:")
print(signature.hex())

# -----
# Step 2: Generate AES Session Key
# -----

aes_key = AESGCM.generate_key(bit_length=256)

# -----
# Step 3: Encrypt Email using AES
# -----

nonce, ciphertext = encrypt_message(
    message,
    aes_key
)

print("\nEncrypted Email:")
print(ciphertext.hex())

# -----
# Step 4: Encrypt AES Key using Receiver Public Key
# -----

encrypted_aes_key = receiver_public.encrypt(
    aes_key,
    padding.OAEP(
        mgf=padding.MGF1(
            algorithm=hashes.SHA256()
        ),
        algorithm=hashes.SHA256(),
        label=None
    )
)

```

```
print("\nEncrypted AES Session Key:")
print(encrypted_aes_key.hex())
```

```
# -----
# RECEIVER SIDE
# -----
```

```
print("\n=====")
print("      RECEIVER SIDE")
print("=====")
```

```
# -----
# Step 5: Decrypt AES Key
# -----
```

```
decrypted_aes_key = receiver_private.decrypt(
    encrypted_aes_key,
    padding.OAEP(
        mgf=padding.MGF1(
            algorithm=hashes.SHA256()
        ),
        algorithm=hashes.SHA256(),
        label=None
    )
)
```

```
# -----
# Step 6: Decrypt Email
# -----
```

```
decrypted_message = decrypt_message(
    nonce,
    ciphertext,
    decrypted_aes_key
)
```

```
print("\nDecrypted Email:")
print(decrypted_message.decode())
```

```
# -----
```

```
# Step 7: Verify Digital Signature
```

```
# -----
```

```
print("\nSignature Verification:")
```

```
if verify_signature(  
    sender_public,  
    decrypted_message,  
    signature  
):
```

```
    print("SUCCESS")  
    print("Message is authentic and has not been modified.")
```

```
else:
```

```
    print("FAILED")  
    print("Message authentication failed.")
```

```
# -----
```

```
# Step 8: Demonstrate Tampering
```

```
# -----
```

```
modified_message = b"""  
From: alice@example.com  
To: bob@example.com  
Subject: Secure Communication
```

```
Hello Bob,  
This message has been modified by an attacker.  
"""
```

```
print("\nTampered Message Verification:")
```

```
if verify_signature(  
    sender_public,  
    modified_message,  
    signature  
):
```

```
    print("SUCCESS")
```

```
else:
```

```
print("FAILED")
print("Tampering detected.")
```

Installation

Install the required Python cryptography package:

```
pip install cryptography
```

Sample Output

```
=====
SIMPLIFIED PGP EMAIL SYSTEM
=====
```

Original Email:

From: alice@example.com
To: bob@example.com
Subject: Secure Communication

Hello Bob,
This is a confidential PGP protected email.

Digital Signature Generated:
[long hexadecimal signature]

Encrypted Email:
[encrypted ciphertext]

Encrypted AES Session Key:
[encrypted RSA ciphertext]

```
=====
RECEIVER SIDE
=====
```

Decrypted Email:

From: alice@example.com

To: bob@example.com
Subject: Secure Communication

Hello Bob,
This is a confidential PGP protected email.

Signature Verification:
SUCCESS
Message is authentic and has not been modified.

Tampered Message Verification:
FAILED
Tampering detected.

Explanation

1. RSA Key Pair

Two key pairs are generated:

Sender:
Private Key → Sign
Public Key → Verify

Receiver:
Private Key → Decrypt
Public Key → Encrypt

2. Digital Signature

The sender signs the original message using the sender's private key.

Message
|
v
SHA-256
|
v
RSA-PSS Signature
|
v

Digital Signature

3. AES Encryption

The actual email is encrypted using a randomly generated AES-256 session key.

AES is efficient and suitable for encrypting the message itself.

4. RSA Encryption

The AES session key is encrypted using the receiver's RSA public key.

Only the receiver's corresponding private key can decrypt the session key.

5. Signature Verification

The receiver uses the sender's public key to verify the signature.

If the message has been modified:

Original Message $\neq$ Modified Message

the signature verification fails.

Security Properties

Security Property	Mechanism Used
Confidentiality	AES-256 encryption
Key protection	RSA-OAEP
Authentication	RSA digital signature
Integrity	Digital signature + SHA-256
Non-repudiation	Digital signature
Secure key exchange	RSA-encrypted AES session key

Test Cases

Test Case	Expected Result
Original message	Signature verification successful
Modified message	Signature verification failed
Correct receiver private key	AES key decrypted successfully
Incorrect private key	Decryption fails
Modified ciphertext	AES-GCM authentication fails
Modified signature	Signature verification fails

Advantages of the Hybrid PGP Approach

1. Performance

AES efficiently encrypts the actual email data.

2. Secure Key Protection

RSA protects the AES session key.

3. Authentication

The digital signature verifies the sender.

4. Integrity

Modification of the signed message causes verification failure.

5. Confidentiality

Only the intended receiver can recover the session key and decrypt the message.

Result

A simplified PGP system was successfully implemented using Python. AES-GCM was used to encrypt the email, RSA-OAEP was used to protect the AES session key, and RSA-PSS with SHA-256 was used for digital signatures.

The original message was successfully decrypted and authenticated. When the message was modified, signature verification failed, demonstrating the ability to detect tampering.

Conclusion

The simplified PGP implementation demonstrates how multiple cryptographic techniques can work together to secure email communication. **AES** provides efficient message encryption, **RSA-OAEP** protects the session key, and **RSA-PSS with SHA-256** provides authentication and integrity through digital signatures.

This hybrid design demonstrates the fundamental security principles behind PGP-style secure email systems and provides **confidentiality, integrity, authentication, and non-repudiation**.

Overall Conclusion

The two coding challenges demonstrate important network-security mechanisms.

Question 1 implements the basic concept of **IPSec Authentication Header**, where HMAC-SHA-256 is used to detect modification of IP packet data.

Question 4 implements a simplified **PGP-style secure email system**, combining symmetric encryption, public-key cryptography, and digital signatures.

Both implementations demonstrate how cryptographic techniques can be combined to protect network communication and electronic messages.